Minimal, Resource-Bounded Runtime Verification for a Safety-Critical Automotive ECU

Damian Navarro
BSc (Hons) Software Development
Institute of Information & Communication Technology (IICT)
Malta College of Arts, Science & Technology (MCAST)
Research Design II (2025/2026)

***Abstract*—This paper investigates minimal, resource-bounded runtime verification for safety-relevant automotive ECU telemetry. A server-side replay pipeline was implemented using LARVA monitors and benchmarked under two regimes: baseline (clean trace) and abnormal (injected violations), with runtime verification toggled on and off on the same endpoint. Four properties were evaluated: voltage out-of-range under running RPM, coolant over-temperature, throttle spike, and load–MAP mismatch. Across 10 measured runs per mode, baseline overhead was 3.41% (115.3 ms RV on vs 111.5 ms RV off), while abnormal overhead was 692.36% (819.3 ms vs 103.4 ms), showing that monitor cost is strongly state-dependent. Injected violations produced high RV-on flag counts while RV-off flags remained zero. Threshold tuning reduced baseline false positives for load–MAP mismatch without removing abnormal detections. A controlled three-variant experiment isolated the dominant overhead source: abnormal overhead moved from 3927.05% (original generated monitor) to 683.79% (logging disabled) and then to 9.39% after removing bad-state stacktrace construction, with CPU-time measurements following the same pattern. These results show that diagnostic generation, rather than guard evaluation, dominates violation-heavy runtime cost.**

I. Introduction

This section presents the research design for the project *Minimal, Resource-Bounded Runtime Verification for a Safety-Critical Automotive ECU*. It follows the structure of the Research Design II write-up, covering the theme and rationale, research positioning, background, hypothesis, and the overall aim and purpose of the study.

A. Title and Programme Details

Module: MCAST IICT – Research Design II (2025/2026)
Student: Damian Navarro
Programme: BSc (Hons) Software Development
Institute: Institute of Information and Communication Technology (IICT), MCAST

B. Chosen Research

The working title of the research is:

> Minimal, Resource-Bounded Runtime Verification for a Safety-Critical Automotive ECU.

The design is grounded in recent work on **automotive cybersecurity testing**, **runtime verification for CPS/IoT**, and **hardware-anchored trust and attestation** [1]–[4].

C. Theme and Topic Rationale

Theme: Safety and cybersecurity assurance for cyber-physical and IoT-like automotive systems.

Specific topic: Designing a minimal set of runtime monitors for a safety-critical ECU (e.g. braking), implementable on constrained hardware while supporting current automotive cybersecurity and homologation requirements [1], [5].

Rationale: New vehicles contain many real-time ECUs connected via CAN-FD buses and a central gateway, and must pass large cybersecurity verification and validation (V&V) test suites under norms such as SAE J3061 and ISO 21434 for homologation [1]. At the same time, IoT-class and automotive controllers are constrained in CPU, memory, power, and area, which limits the applicability of heavyweight security and verification mechanisms [6], [7].

Formal verification and validation surveys for IoT highlight scalability and state-explosion challenges, and recommend lightweight, runtime-oriented approaches for safety- and security-critical systems [5]. Runtime verification (RV) has been shown to be a **pragmatic** technique for CPS and automotive domains, where specifications are represented as automata and monitored during execution [2], [3]. An IoT digital-twin approach demonstrates that only a subset of network traffic needs monitoring to detect communication violations, achieving negligible overhead on constrained devices [3].

To protect keys, firmware, and measurements on resource-constrained platforms, work on hardware trust anchors and remote attestation shows how hardware-based roots of trust and hybrid attestation protocols can provide integrity guarantees while preserving availability [4], [7].

This motivates exploring **how to specify and embed a small, local runtime monitor on a safety-critical ECU** that respects strict resource and timing constraints while contributing to cybersecurity and safety assurance.

D. Positioning and Research Onion

This study investigates a minimal, resource-bounded runtime verification mechanism for a safety-critical automotive Electronic Control Unit (ECU) using a simulation-based experimental design. The methodology is organised using the layers of the Saunders *et al.* research onion [8].

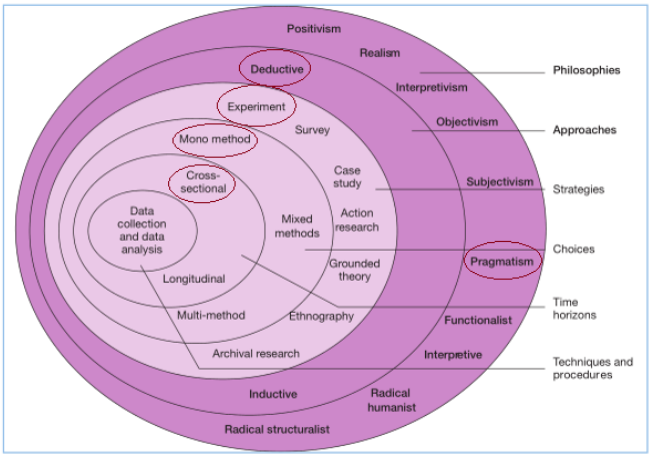

Fig. 1. The research onion (adapted from Saunders *et al.*) [8].

a) Research Philosophy: Pragmatism.: The research is grounded in a **pragmatist** philosophy, focusing on developing a solution that is practically effective under realistic resource constraints. The value of the work is judged by whether the proposed runtime verification approach can be integrated into an automotive ECU context and meet performance and safety requirements, rather than by commitment to a single ontological or epistemological stance.

b) Research Approach: Deductive.: A **deductive** approach is adopted. Existing theory and prior work on runtime verification, safety-critical systems, and real-time resource constraints are used to derive expectations about the behaviour of a minimal monitoring mechanism (e.g., that it should incur low overhead while reliably detecting property violations). These theoretically informed expectations are then tested empirically through controlled experiments in a simulated ECU environment.

c) Methodological Choice: Quantitative Mono Method.: The study employs a **quantitative mono-method** design. All primary data are numerical, capturing performance and correctness characteristics of the monitored system. Key variables include execution time overhead, CPU utilisation, memory footprint, and the rate of correctly detected versus missed property violations. These quantitative measures form the basis for assessing whether the proposed mechanism satisfies the targeted resource bounds and safety criteria.

d) Research Strategy: Experiment.: The chosen research strategy is an **experiment**. A runtime verification component is implemented and integrated with a simulated model of a safety-critical automotive ECU. Experimental runs are conducted under controlled conditions, where input workloads, operating scenarios, and injected faults are systematically varied. A baseline configuration without monitoring is used for comparison, enabling the isolation of the effects of the runtime verification mechanism on system performance and fault detection.

e) Time Horizon: Cross-Sectional.: The research adopts a **cross-sectional** time horizon. Data are collected during a finite experimental campaign consisting of multiple simulation runs, each representing a distinct configuration or scenario. The study does not follow the system over an extended real-world operational period; instead, it captures system behaviour at a specific point in time under a range of controlled experimental conditions.

f) Data Collection and Analysis Procedures.: Data collection is performed by instrumenting the simulated ECU replay service to record, for each run, full-trace elapsed time, process CPU time, heap-delta indicators, throughput, and violation flag totals with RV enabled versus disabled. The replay traces are derived from the Kaggle Car ECU Datalogs dataset [9]. Faults are injected into the abnormal dataset to evaluate whether the monitor reports the expected violation classes.

The collected data are analysed using descriptive statistics (mean, standard deviation, and tail percentiles) and direct comparisons between monitored and unmonitored runs. This quantifies both runtime cost and detection behaviour against the study objectives.

E. Background to the Research Theme

Modern vehicles are complex cyber-physical systems with many ECUs and a central gateway; cybersecurity V&V must cover thousands of network-level test cases to satisfy norms like ISO 21434, influencing how in-vehicle communication is tested and which attack patterns are considered before road release [1]. Parallel developments in IoT show that pervasive connectivity increases the attack surface, while device constraints hinder deployment of traditional security mechanisms [6].

Formal verification and validation surveys for IoT systems emphasise that exhaustive techniques often do not scale, and advocate combining formal specification with lightweight testing and runtime techniques [5]. In intelligent transport and autonomous vehicles, automata-theoretic, context-aware runtime monitors have been shown to detect unsafe events under varying environmental conditions by composing environment models with system Büchi automata [2].

For IoT communications, a digital-twin-based security architecture uses rule-based runtime verification of parametric events (packets) and demonstrates that monitoring only a subset of traffic can still detect protocol violations with negligible overhead on limited devices [3]. Runtime verification has also been generalised in an algebraic framework that views runs and specification automata through monoidal and semiring structures, enabling incremental, quantitative monitoring and case studies in the automotive domain [10].

On the trust side, surveys of IoT security and remote attestation underline the importance of offloading verification to more capable entities while providing device integrity proofs, especially when local resources are scarce [6], [7]. Hybrid remote attestation protocols specifically target IoT

devices, combining randomised memory checks with physical unclonable functions to achieve high availability and low energy usage [4]. Evaluations of hardware trust anchors for automotive applications compare different HTA designs against current and emerging automotive requirements, and identify options that are feasible even for highly constrained controllers [4].

This body of work collectively supports investigating a **minimal, resource-bounded runtime verification design** for a safety-critical ECU that:

- aligns with automotive cybersecurity test practice and regulation [1],
- leverages lightweight RV techniques proven in CPS and IoT contexts [2], [3], [5], and
- is protected and supported by hardware-anchored trust and remote attestation mechanisms [4], [7].

F. Hypothesis

A carefully selected, minimal set of local safety and security properties for a safety-critical automotive ECU can be implemented as runtime monitors with bounded time and memory overhead such that:

- they detect violations relevant to automotive cybersecurity and safety requirements [1]–[3], [5], [11], and
- they remain compatible with the resource and availability constraints of in-vehicle and IoT-class controllers [4], [6], [7].

Formally:

> **H_1:** Minimal, local runtime monitors for a safety-critical ECU can provide meaningful safety and cybersecurity coverage without violating the ECU's real-time and resource constraints.

G. Research Aim and Purpose

Aim: To design and analytically evaluate a minimal, resource-bounded runtime verification monitor for a safety-critical automotive ECU, drawing on automotive cybersecurity V&V practice, CPS/IoT runtime verification techniques, and hardware-anchored trust and attestation mechanisms [1]–[4], [7], [11].

Purpose:

- To derive a method for selecting and formalising a small set of ECU-local properties as automata or algebraic specifications suitable for runtime monitoring under tight resource constraints [2], [3], [5].
- To propose an integration strategy that minimises monitoring overhead through selective observation of events/messages, while still addressing key cybersecurity and safety requirements [1]–[3], [5], [11].
- To outline how hardware trust anchors and remote/hybrid attestation can protect the monitor and its artefacts in automotive-grade, resource-constrained environments [4], [6], [7].

This provides a focused design blueprint that can later be extended and evaluated in broader connected-vehicle and IoT security architectures.

II. Literature Review

Figure 2 summarises how the sources in the research bibliography relate to conceptual themes. Branches group survey, runtime verification, automotive and trust, and methods/tooling; each citation is paired with a short summary (no duplicated sources).

This section reviews how recent studies approach verification, monitoring, and trust for connected, resource-constrained controllers such as automotive ECUs and IoT devices. The discussion is grounded in the sources charted in Fig. 2 [1]–[8], [10]–[19].

A. Methodological themes in related work

Several methodological strands recur across these studies. **Survey and taxonomy approaches** synthesise large bodies of technique. Krichen [5] maps formal verification and validation (FV&V) methods for IoT and highlights where exhaustive analysis gives way to lighter runtime checks. Williams *et al.* [6] catalogue IoT security challenges under emerging connectivity paradigms. **Empirical and industry-oriented testing** is represented by Ekert *et al.* [1], who relate automotive cybersecurity V&V to structured test campaigns and normative expectations. **Runtime monitoring with formal undertones** appears in Zhang *et al.* [2], who use automata-theoretic environment models for context-aware safety monitoring in intelligent transport settings. It also appears in de Hoz Diego *et al.* [3], who combine an IoT digital twin with rule-based runtime verification over parametric network events and show that observing only a subset of traffic can still expose protocol violations with low overhead. **Tool-chain and integration studies** include ROSMonitoring 2.0 [7], which extends runtime verification in ROS to services and ordered topics. Work on hardware trust anchors (HTA) is also relevant: Lorych and Plappert [11] address authentication for updatable IoT devices, while Plappert *et al.* [4] evaluate HTA designs against automotive requirements. Finally, Jakšić *et al.* [10] advance an algebraic view of runtime verification that supports incremental and quantitative monitoring.

B. Source quality note

Academic sources here include peer-reviewed journals (e.g. [1], [2], [4]–[6]), archival conference proceedings (e.g. [11]), and edited volume chapters (e.g. [3]). Workshop and electronic proceedings such as EPTCS [7] are scholarly but differ from flagship journals in review depth and indexing. This review therefore prioritises peer-reviewed evidence while using conference and workshop material where it contributes directly to implementation context.

C. Five recommended peer-reviewed journal articles

For readers prioritising archival journal literature, five papers are especially relevant: (1) Ekert *et al.* [1] on automotive cybersecurity V&V testing; (2) Krichen [5] on FV&V for IoT; (3) Williams *et al.* [6] on IoT security and emerging technologies; (4) Plappert *et al.* [4] on hardware trust anchors

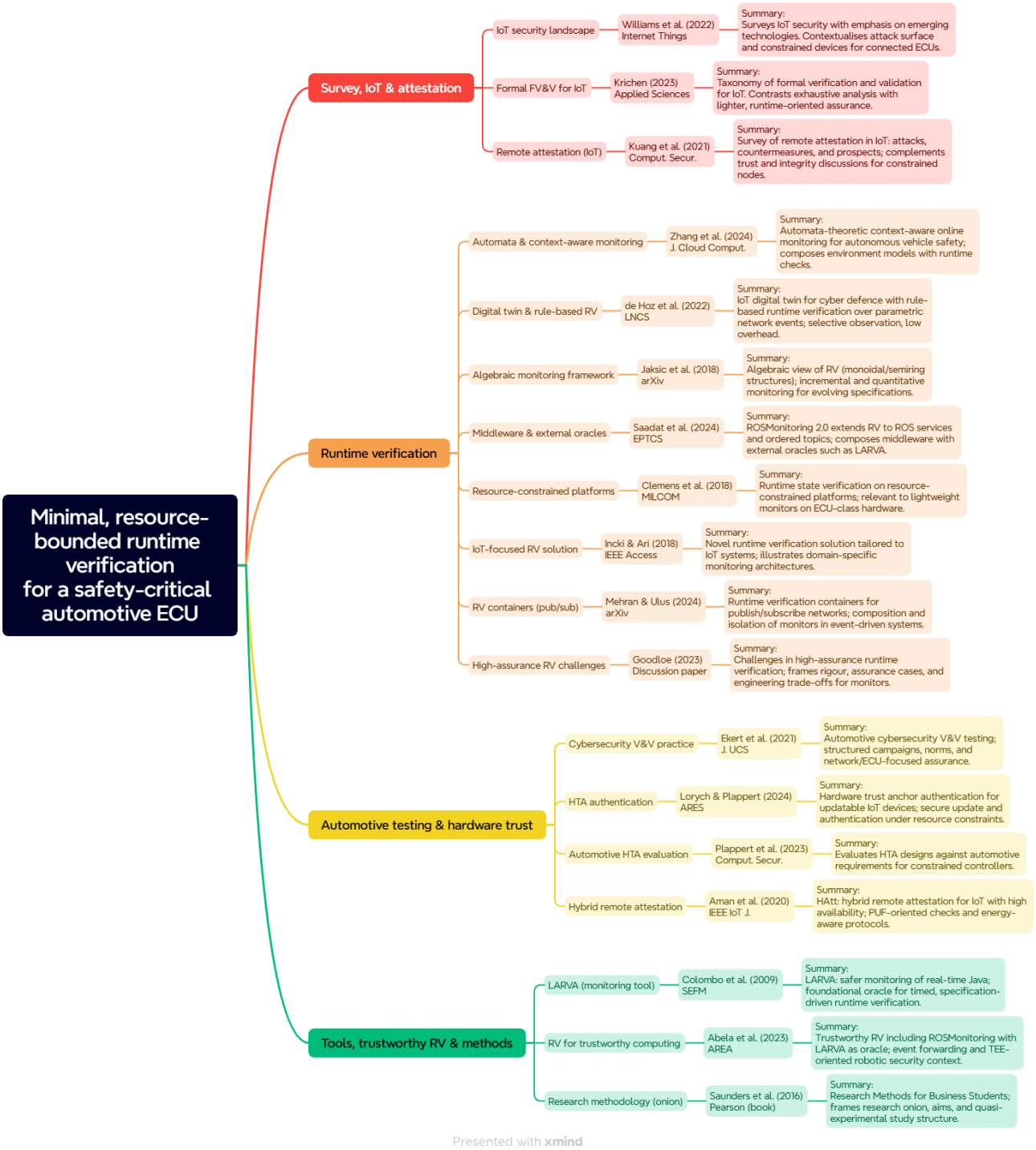

Fig. 2. Literature map covering the full research bibliography (survey/IoT/attestation; runtime verification; automotive testing and hardware trust; tools, trustworthy RV, and methodology). Each citation is paired with a short summary. Sources: Williams *et al.* [6], Krichen [5], Kuang *et al.* [12], Zhang *et al.* [2], de Hoz Diego *et al.* [3], Jakšić *et al.* [10], Saadat *et al.* [7], Clemens *et al.* [13], Incki and Ari [14], Mehran and Ulus [15], Goodloe [16], Ekert *et al.* [1], Lorych and Plappert [11], Plappert *et al.* [4], Aman *et al.* [17], Colombo *et al.* [18], Abela *et al.* [19], Saunders *et al.* [8].

in automotive settings; and (5) Zhang *et al.* [2] on automata-theoretic, context-aware monitoring for autonomous vehicle safety. Together they cover testing practice, formal-method surveys, the security landscape, trust hardware, and concrete monitoring formalisms.

D. Contextualisation for this study

The present research targets a *minimal, resource-bounded* runtime monitor on a safety-related ECU. Within that framing, Ekert *et al.* [1] contextualises the scale and structure of automotive security testing, while Williams *et al.* [6] explains why connectivity increases exposure on constrained nodes. Plappert *et al.* [4] and Lorych and Plappert [11] ground trust mechanisms that could protect monitors and keys where software-only solutions are insufficient. Zhang *et al.* [2] and de Hoz Diego *et al.* [3] motivate automata- and rule-based runtime checks with selective observation, which aligns with limiting bus or signal instrumentation on an ECU. ROSMonitoring 2.0 [7] illustrates how modern tool chains compose middleware events with external oracles; this parallels composing a simulated ECU feed with an external monitor even when the vehicle stack is not ROS-based. Jakšić *et al.* [10] provides useful vocabulary for quantitative or multi-valued verdicts if requirements later move beyond Boolean pass/fail.

E. Critical comparison and knowledge gaps

The surveyed work **complements** but **does not fully cover** an integrated ECU-local runtime verification path. Surveys [5], [6] compare techniques at scale but, by design, stop short of prescribing a minimal monitor set for one braking or powertrain controller. Digital-twin and IoT network studies [3] optimise traffic observation on networked devices, whereas an ECU simulation emphasises *internal* state and sensor plausibility rather than packet subsets. Automata-based context monitoring [2] assumes rich environment models; reproducing that fidelity in a student-grade simulation may trade precision for tractability. ROS-oriented tooling [7] demonstrates composable monitoring, but automotive CAN signal semantics and homologation test cases are not identical to ROS topics. HTA work [4], [11] addresses roots of trust and update authentication, yet leaves open *which* temporal properties an on-device monitor should enforce under strict CPU/RAM caps. The algebraic framework [10] generalises monitoring semantics but does not resolve deployment questions for legacy ECU binaries.

Knowledge gaps salient for this project include: (i) lack of published, reproducible pipelines that pair *lightweight* ECU signal simulation with declarative monitors such as LARVA for automotive-grade properties; (ii) fragmented guidance on calibrating thresholds (e.g. temperature, RPM) against both safety and cybersecurity abuse cases; (iii) limited reporting on measured overhead when co-locating monitors with attestation or secure boot on class-A/B automotive microcontrollers. Addressing (i)–(iii) experimentally would extend the literature beyond the surveyed sources while building directly on their methodological lessons.

III. Research Methodology

This section describes the empirical setup used to evaluate a minimal runtime verification layer over an automotive ECU telemetry stream. The approach is **trace-driven** and **monitor-centric**: recorded ECU time-series are replayed server-side, and an external runtime verifier checks those streams against formalised properties. The design follows a quasi-experimental logic [8]: replay traces act as controlled behaviour sources, while monitors provide repeatable verdicts on normality versus violation. The deployment assumption is that an IoT/ECU endpoint forwards telemetry records to a server-side analysis service; hence measured overhead reflects server-side monitoring cost for ingested records, not direct on-device ECU execution overhead.

A. Overall approach

The methodology proceeds in four stages: (1) prepare a cleaned ECU dataset with stable schema and timestamps (sourced from the Kaggle Car ECU Datalogs dataset [9]); (2) construct an abnormal variant by injecting controlled violations into existing rows while preserving file size; (3) deploy **LARVA** [18] in a Spring-based server endpoint that processes each sample and raises monitor states; and (4) benchmark latency and throughput by replaying the same input under RV enabled versus disabled execution paths.

Abela *et al.* [19] describe a prototype that couples LARVA with ROSMonitoring for robotic systems:

> As a step towards deploying RV-TEE within robotic systems, we have developed a prototype which combines the RV tool which we have used in our previous works, Larva, with ROSMonitoring to successfully monitor a ROS-based system. Fig. 4 shows how ROSMonitoring listens for relevant events (also known as topics) occurring within the ROS application. These are then forwarded to the Larva monitor, which in turn can send back commands to the system being monitored. As ROSMonitoring is agnostic to the chosen verification system (also referred to as oracle) by design, it was not difficult to combine it with Larva.

—Abela *et al.* [19]

In this study, a server-side CSV replay service supplies the ordered event stream (analogous to middleware or bus traffic), and LARVA acts as the oracle, without mandating a ROS deployment.

B. Instrumentation: LARVA and flags

LARVA supports specification-driven monitoring of timed behaviour [18]. Each *flag* below is implemented as a monitor property that raises a violation when its predicate becomes false. The evaluated assertion flags are:

1) **Voltage out of range under running RPM.** Trigger when RPM is above idle threshold and battery voltage is outside calibrated bounds.
2) **Coolant too high.** Trigger when coolant temperature exceeds a configured thermal safety limit.
3) **Throttle spike.** Trigger when throttle delta exceeds a threshold within a short runtime interval.
4) **Load–MAP mismatch.** Trigger when absolute disagreement between Load and MAPSource exceeds tolerance.

Each flag is evaluated on replayed traces; the abnormal dataset injects approximately 20% violation rows distributed across the four flags to validate that the corresponding LARVA specification fires, while benign traces provide a baseline for false-positive checks.

C. Benchmark protocol

Two benchmark regimes are executed with identical machine and JVM conditions: **baseline** (clean trace) and **abnormal** (injected trace). In both regimes, the same endpoint and CSV shape are used, and only RV instrumentation is toggled through a per-request parameter (`rvEnabled=true/false`) and the global server setting (`rv.enabled`). Each regime runs warmup iterations followed by ten measured trials per mode. Reported metrics include rows processed, elapsed time, throughput (rows/s), average, standard deviation, p95/p99 latency, and overhead. Each elapsed-time value represents end-to-end processing of the full input trace per request (68,433 rows), not per-row latency:

$$\text{overhead}(\%) = \frac{\bar{t}_{\text{RV on}} - \bar{t}_{\text{RV off}}}{\bar{t}_{\text{RV off}}} \times 100.$$

D. Research questions and validation

The following questions guide evaluation: (RQ1) Do the four selected properties detect the injected abnormal classes with high recall? (RQ2) What is the observed monitoring overhead relative to non-instrumented processing on the same server path? (RQ3) How stable are latency and throughput across repeated runs (mean, standard deviation, and tail percentiles)? Answers are sought by structured log analysis and repeated trials, consistent with cybersecurity V&V emphasis on repeatable test cases [1].

E. Limitations

Results apply to the simulation and property set instantiated here; they do not replace homologation testing or formal verification of production ECU binaries. Future work could couple the same monitors with recorded CAN traces or hardware-in-the-loop benches.

IV. Findings & Discussion of Results

This section reports the experimental outcomes of the server-side ECU trace replay with LARVA instrumentation toggled on/off. Results are drawn from the benchmark session with tuned `loadMapMismatch` threshold (`|Load-MAPSource| > 60`), using 3 warmup runs and 10 measured runs per mode on the same machine and JVM conditions. The experiment assumes telemetry is transmitted from an IoT/ECU endpoint to a server-side verification API.

A. Result presentation

Two regimes were evaluated:

- **Baseline regime:** clean trace (`20180713-home2mimos_clean.csv`).
- **Abnormal regime:** injected trace (`20180713-home2mimos_clean_abnormal.csv`) with approximately 20% violation rows across four flags.

Table I summarises core timing and throughput metrics. Standard deviation is reported with each average to capture run-to-run variability. All elapsed-time values reported in this section correspond to full-trace processing per request (68,433 rows).

TABLE I
RUNTIME VERIFICATION OVERHEAD SUMMARY (10 MEASURED RUNS PER MODE)

Metric	Baseline	Abnormal
RV ON elapsed (ms)	115.3 ± 6.04	819.3 ± 19.29
RV OFF elapsed (ms)	111.5 ± 20.03	103.4 ± 1.71
RV ON throughput (rows/s)	594,937.4 ± 30,089.0	83,567.3 ± 1,940.6
RV OFF throughput (rows/s)	626,961.4 ± 82,253.9	661,990.5 ± 10,913.9
Overhead (%)	3.41	692.36

Tail latency also reflects this split behaviour. For baseline, RV ON p95/p99 are 124.40/127.28 ms, while RV OFF p95/p99 are 145.75/161.95 ms. For abnormal traces, RV ON p95/p99 are 850.80/856.56 ms, whereas RV OFF remains at 106.00/106.00 ms.

B. Analysis and interpretation

RQ1 (detection of injected abnormalities). The abnormal regime produced high RV ON flag totals (`coolantTooHigh`=34210, `voltageOutOfRange`=34200, `throttleSpike`=34220, `loadMapMismatch`=34230), while RV OFF flags remained zero. This indicates that violation reporting is coupled to monitor execution as intended and that injected abnormal classes are detected consistently.

RQ2 (monitoring overhead). Baseline overhead is low (3.41%), suggesting that monitor integration is feasible for non-adversarial trace processing on the tested server path. Abnormal overhead remains substantial (692.36%), showing that violation-heavy execution paths still dominate runtime even after reducing logging overhead. In other words, monitor cost is not constant; it is strongly state-dependent. These overhead values should be interpreted as server-side verification cost under a telemetry-upload architecture.

RQ3 (stability and threshold sensitivity). Standard deviations are small relative to means in most cases, indicating repeatable behaviour. After tuning `loadMapMismatch` from 30 to 60, baseline false positives reduced (aggregate baseline `loadMapMismatch` decreased compared with pre-tuning runs), while abnormal detection remained strong. This supports threshold calibration as a practical control for false-positive pressure.

C. Controlled root-cause experiment

To isolate the source of abnormal-path overhead, an additional controlled comparison was executed on three monitor variants under identical benchmark settings (1 warmup, 5 measured runs): (i) original generated monitor, (ii) logging

disabled, and (iii) logging disabled plus removal of bad-state stacktrace/string construction. Timing results are summarised in Table II, and resource metrics are summarised in Table III.

TABLE II
ABNORMAL-PATH OVERHEAD ACROSS MONITOR VARIANTS (CONTROLLED 3-WAY COMPARISON)

Variant	Base O/H (%)	Abn O/H (%)	Abn RV ON (ms)
Original	6.43	3927.05	4824.4
Logging disabled	14.61	683.79	851.2
No stacktrace	6.75	9.39	125.8

This progression shows that the dominant abnormal overhead originates from generated diagnostic behaviour in bad-state handling (especially stacktrace-backed state-string construction), not from the core guard checks themselves.

TABLE III
RESOURCE METRICS ACROSS MONITOR VARIANTS (CONTROLLED 3-WAY COMPARISON, ABNORMAL RUNS)

Variant	On CPU	Off CPU	On Heap Δ	Off Heap Δ
Original	3562.5	125.0	-30.59	1.85
Logging disabled	850.0	100.0	-10.62	12.19
No stacktrace	125.0	112.5	13.97	-26.13

CPU time follows the same trend as elapsed time, confirming that compute spent inside monitor-side diagnostics is the dominant contributor in abnormal paths. Heap delta is less stable across variants because Java GC activity during full-trace requests can produce negative or positive net deltas; therefore, heap delta is interpreted as a coarse indicator, while CPU and elapsed-time comparisons provide the primary evidence.

D. Comparison with related work and quality considerations

The observed trade-off aligns with prior RV literature: runtime monitoring provides actionable safety/cybersecurity visibility, but overhead depends on property design, trigger frequency, and deployment context [2], [5], [10]. The clear difference between baseline and abnormal runtime also mirrors broader automotive V&V concerns, where worst-case and abuse-case behaviour must be measured rather than assumed from nominal conditions [1].

Validity. Internal validity is strengthened by same-endpoint comparisons, warmup before measured runs, and on/off toggling as the only intended treatment factor. Construct validity is supported by explicit metrics (elapsed time, throughput, p95/p99, overhead) and deterministic replay inputs.

Reliability. Repeated 10-run measurements with reported standard deviations indicate reproducibility under stable local conditions.

Generalizability. External validity remains limited to this server-side replay implementation and selected property set. Results should not be extrapolated directly to production ECUs without hardware-in-the-loop or in-vehicle evaluation.

V. CONCLUSION

This study evaluated a minimal runtime verification pipeline for automotive ECU telemetry replay, using LARVA-driven monitoring over four properties (`voltageOutOfRange`, `coolantTooHigh`, `throttleSpike`, `loadMapMismatch`). The empirical objective was to quantify monitoring overhead and detection behaviour under controlled baseline and abnormal traces.

The findings indicate that: (i) injected abnormal classes are detected consistently when RV is enabled and are not reported when RV is disabled; (ii) monitoring overhead is low for nominal traces (3.41%) but substantial in violation-heavy scenarios (692.36%); and (iii) threshold tuning reduces false positives without collapsing abnormal detection performance. The controlled three-variant comparison showed that abnormal overhead is mainly driven by generated bad-state diagnostics: 3927.05% (original generated) decreased to 683.79% (logging disabled) and then to 9.39% after removing stacktrace-based bad-state string construction, with CPU-time measurements showing the same trend. The hypothesis is therefore **partially supported**: useful safety/cybersecurity coverage is achievable with bounded overhead for nominal workloads, and worst-case overhead can be reduced substantially through targeted monitor-code optimisation.

Methodological limitations should be acknowledged. First, evaluation is trace-driven and server-side, not hardware-in-the-loop on target automotive microcontrollers. The assumed deployment is that IoT/ECU records are forwarded to a server for verification, so measured overhead is server-path overhead rather than on-device monitor overhead. Second, the selected property set is intentionally small and does not cover all dynamics of production powertrain/braking stacks. Third, the abnormal dataset is synthetic (though controlled), and therefore may not represent full real-world fault diversity.

Future work should focus on four directions. (1) Move from server replay to ECU-representative hardware and measure CPU/memory budgets directly. (2) Introduce adaptive/specification-level optimisations (e.g., event filtering, staged monitors, lower-cost violation handling) to reduce violation-path latency. (3) Expand validation with recorded CAN/FlexRay traces and hardware-in-the-loop campaigns. (4) Integrate trust primitives (e.g., attestation and hardware trust anchors) with monitor lifecycle management to align runtime verification with broader automotive cybersecurity assurance frameworks [1], [4], [11].

REFERENCES

[1] D. Ekert, J. Dobaj, and A. Salamun, "Cybersecurity verification and validation testing in automotive," *J. Univers. Comput. Sci.*, vol. 27, pp. 850–867, 2021.

[2] Y. Zhang, S. Xu, H.-Y. Chen, U. Bhatti, and M. Huang, "Context-aware environment online monitoring for safety autonomous vehicle systems: an automata-theoretic approach," *J. Cloud Comput.*, vol. 13, 2024.

[3] J. de Hoz Diego, A. Temperekidis, P. Katsaros, and C. Konstantinou, "An iot digital twin for cyber-security defence based on runtime verification," in *Software Engineering and Formal Methods*, ser. LNCS. Springer, 2022, pp. 556–574.

[4] C. Plappert, D. Lorych, M. Eckel, L. Jäger, A. Fuchs, and R. Heddergott, "Evaluating the applicability of hardware trust anchors for automotive applications," *Comput. Secur.*, vol. 135, p. 103514, 2023.

[5] M. Krichen, "A survey on formal verification and validation techniques for internet of things," *Applied Sciences*, 2023.

[6] P. Williams, I. K. Dutta, H. G. Daoud, and M. Bayoumi, "A survey on security in internet of things with a focus on the impact of emerging technologies," *Internet Things*, vol. 19, p. 100564, 2022.

[7] M. G. Saadat, A. Ferrando, L. A. Dennis, and M. Fisher, "Rosmonitoring 2.0: Extending ros runtime verification to services and ordered topics," *EPTCS*, 2024.

[8] M. Saunders, P. Lewis, and A. Thornhill, *Research Methods for Business Students*, 7th ed. Harlow: Pearson, 2016.

[9] gxkok21, "Car ecu datalogs," [Online]. Available: https://www.kaggle.com/datasets/gxkok21/car-ecu-datalogs, 2024, accessed: 2026-05-09.

[10] S. Jakšić, E. Bartocci, R. Grosu, and D. Ničković, "An algebraic framework for runtime verification," *arXiv*, 2018.

[11] D. Lorych and C. Plappert, "Hardware trust anchor authentication for updatable iot devices," in *Proceedings of the 19th International Conference on Availability, Reliability and Security*, 2024.

[12] B. Kuang, A. Fu, W. Susilo, S. Yu, and Y. Gao, "A survey of remote attestation in internet of things: Attacks, countermeasures, and prospects," *Computers & Security*, 2021.

[13] J. Clemens, R. Pal, and B. Sherrell, "Runtime state verification on resource-constrained platforms," in *Milcom 2018 Track 3 – Cyber Security and Trusted Computing*, 2018.

[14] K. Incki and I. Ari, "A novel runtime verification solution for iot systems," *IEEE Access*, 2018.

[15] A. Mehran and D. Ulus, "Runtime verification containers for publish/subscribe networks," *arXiv*, 2024.

[16] A. Goodloe, "Challenges in high-assurance runtime verification," *none*, 2023.

[17] M. N. Aman, M. H. Basheer, S. Dash, J. W. Wong, J. Xu, H. W. Lim, and B. Sikdar, "Hatt: Hybrid remote attestation for the internet of things with high availability," *IEEE Internet of Things Journal*, 2020.

[18] C. Colombo, G. J. Pace, and G. Schneider, "LARVA: Safer monitoring of real-time java programs (tool paper)," in *Proc. 7th IEEE Int. Conf. Software Engineering and Formal Methods (SEFM)*. IEEE Computer Society, 2009, pp. 33–37.

[19] R. Abela, C. Colombo, A. Curmi, M. Fenech, M. Vella, and A. Ferrando, "Runtime verification for trustworthy computing," *AREA 2023*, 2023.